Supplementary Information

A flexible modelling framework for estimating thermal sensitivity across life

Daniel W.A. Noble*[1], Pieter A. Arnold[1], Patrice Pottier[2]

[1] *Division of Ecology and Evolution, Research School of Biology, Australian National University, Canberra, Australia*

[2] *Department of Biological and Environmental Sciences, University of Gothenburg, Gothenburg, Sweden*

2026-05-08

Table of contents

1 A Tutorial with Simulations

This section is a guided tutorial using simulated data. We show, step by step, how the Bayesian hierarchical 4PL — under both binomial sampling and the more realistic beta-binomial (overdispersed) case – can recover thermal sensitivity (z) and $CT_{max_{1hr}}$ from the single joint model, yet provide substantial downstream benefits in terms of propagating uncertainty and giving more flexibility. We then complement the simulation with a few case studies which walk through real data that is presented within the main manuscript in other sections.

*Corresponding author: daniel.noble@anu.edu.au

1.0.1 *Simulating Data*

1.0.1.1 Step 1 — Setting up parameters

We choose values that might be typical of a thermal mortality assay. The four 4PL parameters are the lower and upper asymptotes (ℓ, u), the slope on the $\log_{10}$-time axis (k), and a midpoint that varies linearly with temperature: $\text{mid}(T) = \beta_0 + \beta_1(T - \bar{T})$. Each value below has a clear biological meaning.

```
true_params <- list(
  ell      = 0.03,   # 3% of individuals survive even at the longest exposures
  u        = 0.97,   # 97% are alive at very short exposures
  k        = 8,      # steepness of the survival drop on the log10(t) axis
  m_beta0  = 1.5,    # midpoint at the grand-mean temperature: ~31.6 min
  m_beta1  = -0.15,  # midpoint drops 0.15 log10-min per °C; z = -1/beta1   6.7 °C
  T_bar    = 34      # grand-mean temperature
)
```

The implied thermal sensitivity is $z = -1/\beta_1 \approx 6.7°\text{C}$ — the temperature change that shifts LT50 by a factor of 10. From the same parameters we can also compute the simulation truth for the uncentred TDT intercept $\alpha = \beta_0 + \frac{1}{k}\log\frac{u-0.5}{0.5-\ell} - \beta_1\bar{T}$ and the critical thermal limit at 1 hour $CT_{max_{1hr}} = (\log_{10} 60 - \alpha)/\beta_1$ — both approaches below will be checked against these known values.

Recall that we can calculate $ $, z and $CT_{max_{1hr}}$ from the true parameters using the equations in the main manuscript. We'll do that now so we have a comparator for later.

```
true_z      <- -1 / true_params$m_beta1
true_alpha  <- true_params$m_beta0 +
  (1 / true_params$k) *
  log((true_params$u - 0.5) / (0.5 - true_params$ell)) -
  true_params$m_beta1 * true_params$T_bar
true_CTmax  <- (log10(60) - true_alpha) / true_params$m_beta1
```

1.0.1.2 Step 2 — Simulated study design

We mimic a textbook TDT experiment: a grid of five assay temperatures crossed with six log-spaced exposure durations, repeated 30 times per cell, with 30 individuals tested per replicate. That gives 900 trials and 27,000 individuals — large enough to recover the parameters with negligible uncertainty, but small enough to fit in a few minutes. Of course, in practice we will not have this much data as will be seen in the case studies, but simulations are a good starting point to understand the models being fit and the conversions.

```
temps     <- c(30, 32, 34, 36, 38)          # 5 assay temperatures (°C)
durations <- c(1, 5, 15, 45, 135, 405)      # 6 durations (minutes, log-spaced)
n_rep     <- 30                             # 30 replicates per cell
n_per_rep <- 30                             # 30 individuals per replicate
```

```r
design <- tidyr::expand_grid(
  T   = temps,
  t   = durations,
  rep = seq_len(n_rep)
) |>
  dplyr::mutate(
    log10_t = log10(t),
    T_c     = T - true_params$T_bar,                          # mean-centred T
    mid     = true_params$m_beta0 + true_params$m_beta1 * T_c,  # mid(T)
    p_true  = true_params$ell +
      (true_params$u - true_params$ell) /
      (1 + exp(true_params$k * (log10_t - mid))),
    n = n_per_rep
  )
```

1.0.1.3 Step 3 — Look at the truth before simulating

Before generating noisy data, let's plot the *true* survival surface (Figure S S1) so we know what the model is being asked to recover. Each line is the 4PL at one assay temperature, plotted against $\log_{10}$ exposure time.

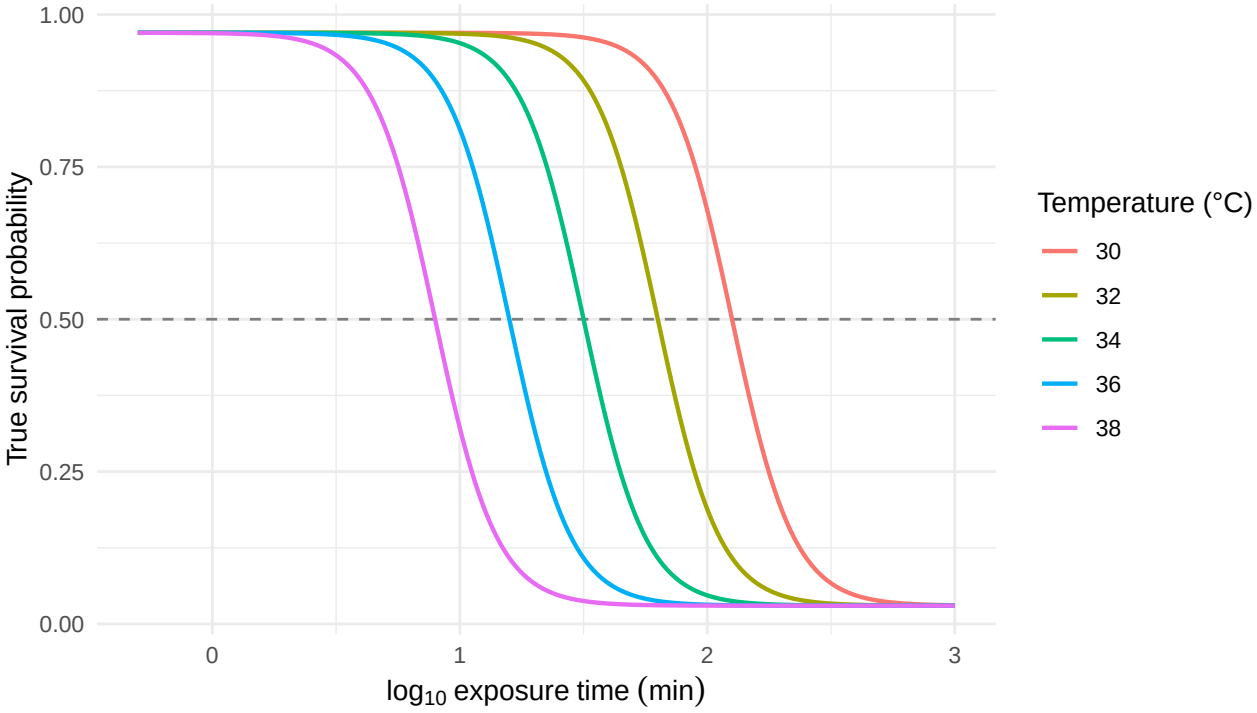

Figure S S1: The known 4PL dose-response surface used to generate the simulated data, evaluated on a dense time grid at each of the five assay temperatures. The horizontal line marks 50% survival; where each curve crosses it gives the true LT50 at that temperature.

Notice the curves shift left as temperature rises: hotter temperatures kill faster, so the LT50 is

reached at shorter durations. That horizontal shift in midpoints is exactly what β_1 encodes.

1.0.1.4 Step 4 — Generate noisy data: binomial and beta-binomial

Real survival counts have at least two flavours of noise. **Binomial** sampling is the textbook case: each replicate's survival count is a draw from Binomial(n, p) with p from the 4PL surface. **Beta-binomial** sampling adds *overdispersion* — replicate-to-replicate variability in p that the binomial cannot capture, e.g., from cohort effects or unmodelled heterogeneity. Real TDT data almost always show overdispersion so in many cases the beta binomial will be a better fit.

```
# (a) Binomial: y ~ Binomial(n, p_true)
sim_bin <- design |>
  dplyr::mutate(y = rbinom(dplyr::n(), size = n, prob = p_true))

# (b) Beta-binomial: replicate-level p drawn from Beta(p*phi, (1-p)*phi)
phi <- 5  # smaller phi = more overdispersion
sim_bb <- design |>
  dplyr::mutate(
    p_draw = rbeta(dplyr::n(), p_true * phi, (1 - p_true) * phi),
    y      = rbinom(dplyr::n(), size = n, prob = p_draw)
  )
```

1.0.2 *The classical two-stage TDT pipeline*

We first run the simulated data through the **classical two-stage TDT pipeline** (Ørsted *et al.* 2022; Rezende *et al.* 2014), which is dominant in the thermal-tolerance literature.

1. **Stage 1.** At each assay temperature, fit a logistic dose-response curve to the count data and read off a point estimate of $\log_{10} \mathrm{LT50}_T$.
2. **Stage 2.** Regress those Stage-1 point estimates on assay temperature with ordinary least squares, then derive $z = -1/\beta_1$ and $CT_{max_{1hr}} = (\log_{10} 60 - \alpha)/\beta_1$ from the fitted line.

1.0.2.1 Stage 1 — Dose-response curves at each temperature

We fit a binomial GLM with a logit link separately at each assay temperature. This is the most-common Stage-1 specification in the TDT literature: a two-parameter logistic with asymptotes constrained to 0% and 100%. The midpoint $\log_{10} \mathrm{LT50}_T = -\beta_0/\beta_1$ falls out of the GLM's two coefficients.

```
fit_stage1 <- function(sim_data) {
  sim_data |>
    dplyr::group_by(T) |>
    dplyr::group_modify(function(d, key) {
      f  <- glm(cbind(y, n - y) ~ log10_t, data = d, family = binomial())
      co <- coef(f)
      tibble::tibble(log10_lt50 = as.numeric(-co[1] / co[2]))
    }) |>
```

Table S S1: Per-temperature Stage-1 estimates of $\log_{10}$ LT50 from a binomial GLM with logit link, fit separately at each of the five assay temperatures. Truth values are the simulated midpoints (which coincide with $\log_{10}$ LT50 here because the simulation asymptotes are near 0 and 1).

T (°C)	Truth log10(LT50)	Binomial log10(LT50)	Beta-binomial log10(LT50)
30	2.1	2.061	2.082
32	1.8	1.795	1.8
34	1.5	1.48	1.528
36	1.2	1.202	1.195
38	0.9	0.907	0.954

```
    dplyr::ungroup()
}

stage1_bin <- fit_stage1(sim_bin)
stage1_bb  <- fit_stage1(sim_bb)
```

With the simulation's true asymptotes near 0 and 1, the 2PL's forced-asymptote constraint introduces negligible bias here, so each Stage-1 estimate sits close to the simulation truth.

1.0.2.2 Stage 2 — log-linear TDT regression

Stage 2 regresses Stage-1 $\log_{10}$ LT50 on assay temperature with ordinary least squares. The classical TDT quantities — $z = -1/\beta_1$ and $CT_{max_{1hr}} = (\log_{10} 60 - \alpha)/\beta_1$ — fall out of the fitted line.

```
fit_stage2 <- function(stage1) {
  fit2 <- lm(log10_lt50 ~ T, data = stage1)
  co   <- coef(fit2)

  tibble::tibble(
    quantity = c("z (°C)", "CTmax at 1 hr (°C)"),
    est      = c(as.numeric(-1 / co["T"]),
                 as.numeric((log10(60) - co["(Intercept)"]) / co["T"]))
  )
}

ts_bin <- fit_stage2(stage1_bin)
ts_bb  <- fit_stage2(stage1_bb)
```

Figure S S2 visualises the Stage-2 fit. Each point is one Stage-1 $\log_{10}$ LT50 estimate from Table S S1; the solid line is the OLS regression through them. The slope is β_1, from which $z = -1/\beta_1$ — i.e., the temperature increase that causes LT50 to change by a factor of 10. The line's height at $\log_{10} 60 \approx 1.78$ (1 hour) is at the temperature equal to $CT_{max_{1hr}}$ — the vertical dotted line

drops from this intersection straight to the temperature axis to mark it. The dashed grey line is the simulation truth.

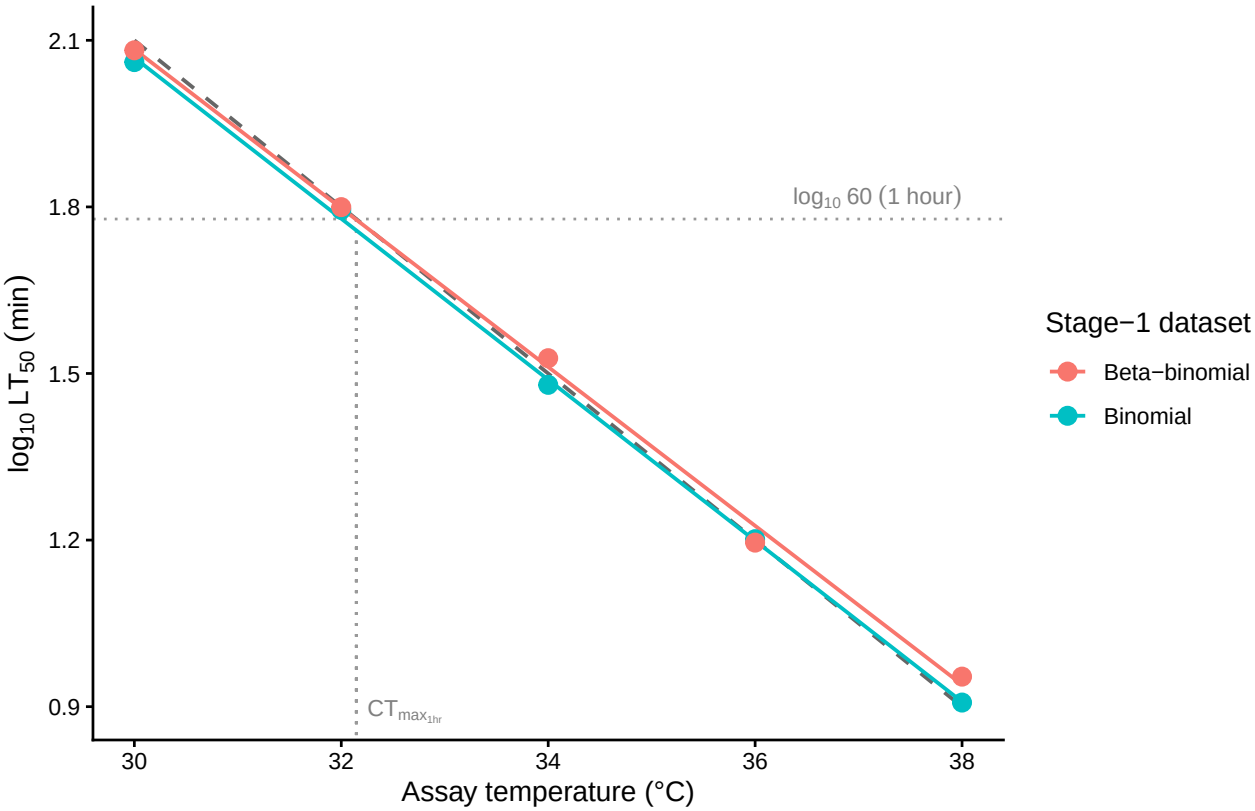

Figure S S2: Stage-2 OLS regression of $\log_{10}$ LT50 on assay temperature, fit separately to the binomial and beta-binomial simulated datasets. Points are the Stage-1 per-temperature LT50 estimates; solid coloured lines are the OLS fits; the dashed grey line is the simulation truth. The slope of each line equals β_1, from which $z = -1/\beta_1$. The horizontal dotted line marks $\log_{10} 60$ (1 hour); the vertical dotted line drops from where the truth crosses 1 hour straight down to the temperature axis, marking $CT_{max_{1hr}}$ visually.

This is the entire two-stage workflow. The point estimates that come out — z and $CT_{max_{1hr}}$ — are what the field reports. We carry them forward (`ts_bin` and `ts_bb`) for the side-by-side comparison once we have the joint Bayesian results.

> **!** Important
>
> Two-stage steps often ignore uncertainty in estimates in both fitting and error propagation. Uncertainty in estimates can be propagated using the SE's and the delta method, but only at each individual stage.

1.0.3 *A joint Bayesian 4PL*

Instead of fitting a separate logistic at each assay temperature and then regressing the LT50s, the joint fit estimates every parameter — ℓ, u, k, β_0 and β_1 — in a single hierarchical model on the raw counts. We now use the same simulated data to derive the z and $CT_{max_{1hr}}$ from the posterior, in one model rather than two.

1.0.3.1 Specifying the 4PL in `brms`

The model has two pieces. The first is the 4PL itself — survival probability as a function of $\log_{10}$ exposure time:

$$p_{ij} \;=\; \ell \;+\; \frac{u-\ell}{1+\exp\!\big(k\,(\log_{10} t_{ij} - \mathrm{mid}_{ij})\big)}, \tag{S1}$$

where ℓ and u are the lower and upper asymptotes, k is the slope on the $\log_{10} t$ axis, and mid_{ij} is the value of $\log_{10} t$ at which the curve crosses the midpoint between ℓ and u. The second piece lets the midpoint vary linearly with mean-centred temperature:

$$\mathrm{mid}_{ij} \;=\; \beta_0 + \beta_1\,(T_{ij} - \bar{T}), \tag{S2}$$

so that β_1 encodes how fast the dose-response curve shifts on the time axis as assay temperature changes; $\bar{T}$ is the grand-mean temperature, included so that β_0 is interpretable as the midpoint at the average assay temperature.

In `brms` non-linear syntax these two pieces become one formula and four sub-formulas — one for each parameter in the 4PL model:

```
# Soft constraints to keep the parameters in their natural ranges.
# These are not informative priors - just the bounds the parameters
# must obey to be biologically meaningful.
priors_4pl <- c(
  prior(beta(1, 5),    nlpar = "ell", lb = 0, ub = 1),
  prior(beta(5, 1),    nlpar = "u",   lb = 0, ub = 1),
  prior(normal(0, 10), nlpar = "k",   lb = 0),
  prior(normal(0, 5),  nlpar = "mid")
)

bform_bin <- bf(
  y | trials(n) ~ ell + (u - ell) / (1 + exp(k * (log10_t - mid))),
  ell ~ 1, u ~ 1, k ~ 1, mid ~ T_c,
  nl = TRUE,
  family = binomial(link = "identity")
)

# Same model, beta-binomial likelihood for the overdispersed dataset.
bform_bb <- bf(
```

```
  y | trials(n) ~ ell + (u - ell) / (1 + exp(k * (log10_t - mid))),
  ell ~ 1, u ~ 1, k ~ 1, mid ~ T_c,
  nl = TRUE,
  family = beta_binomial(link = "identity")
)
```

The first line in `bf()` is Equation S S1. The next four lines say (i) the asymptotes and slope are simply scalars and we estimate an overall intercept for each (`ell ~ 1`, `u ~ 1`, `k ~ 1`) and (ii) the midpoint depends linearly on mean-centred temperature (`mid ~ T_c`), recovering Equation S S2. This is the entire model.

> **!** Important
>
> The model we fit is rather simple — it can be extended in several ways. For example, random effects can be added to any of the four 4PL parameters; for instance, `mid ~ T_c + (1 | day)` allows the midpoint to vary across assay days or batches. Temperature can also be a predictor for the other 4PL parameters — adding `k ~ T_c`, for instance, would let the slope of the dose-response curve itself depend on temperature. Random effects in the joint fit also enable a much more detailed variance decomposition than the two-stage pipeline can deliver. We'll explore more of this in Section 1.0.6.

1.0.3.2 Fitting the model

Now that we have the model formula set up and priors we can fit the model. We pass `file = ...` and `file_refit = "on_change"` directly to `brm()`. This tells `brms` to save the fit to disk on first run and to reload it on subsequent runs unless the model or data has changed.

```
# Disable Quarto chunk caching: brms's `file = ...` / `file_refit = "on_change"`
# already handles refitting on formula/data changes. Quarto's own cache would
# hold a stale fit object across renders.

fit_bin <- brm(
  bform_bin, data = sim_bin, prior = priors_4pl,
  chains = 4, iter = 2000, cores = 4, seed = 123,
  file = file.path(models_dir, "sim_4pl_binomial"),
  file_refit = "on_change"
)

fit_bb <- brm(
  bform_bb, data = sim_bb, prior = priors_4pl,
  chains = 4, iter = 2000, cores = 4, seed = 123,
  file = file.path(models_dir, "sim_4pl_betabinomial"),
  file_refit = "on_change"
)
```

We can also get an idea of how much variation our model explains that considers all sources of relevant variation.

Table S S2: Posterior medians and 95% credible intervals for the four 4PL parameters and the temperature slope, compared to the known truth used to generate the simulated data. Recovery is judged successful when the credible interval covers the truth.

Parameter	Truth	Bin. median	Bin. lower	Bin. upper	BB median	BB lower	BB upper
ell	0.03	0.0288	0.0252	0.0329	0.0295	0.0223	0.0381
u	0.97	0.9675	0.9637	0.9709	0.9679	0.9598	0.9743
k	8	8.3196	7.9162	8.7628	7.5159	6.8119	8.3283
mid (intercept)	1.5	1.5012	1.4927	1.5095	1.5157	1.4951	1.5366
mid (slope)	-0.15	-0.15	-0.1527	-0.1471	-0.1474	-0.1544	-0.1404

```
# Obtain the Bayes R2 for each model
brms::bayes_R2(fit_bin)
```

```
    Estimate    Est.Error     Q2.5     Q97.5
R2 0.9875292 0.0001264011 0.987227 0.9877159
```

```
brms::bayes_R2(fit_bb)
```

```
    Estimate  Est.Error      Q2.5     Q97.5
R2 0.9263016 0.00105021 0.9238746 0.9280369
```

1.0.3.3 Checking parameter recovery

We can now check how well our fitted model recovered the true parameters that generated the data. The most direct check is to compare posterior medians (with 95% credible intervals) to the known truth (Table S S2). We have big sample sizes here so our values should be pretty close to our true values.

We can also overlay the posterior fitted curves on the simulated data (Figure S S3). If the model has recovered the true surface, the fitted lines should track the simulated proportions at every assay temperature.

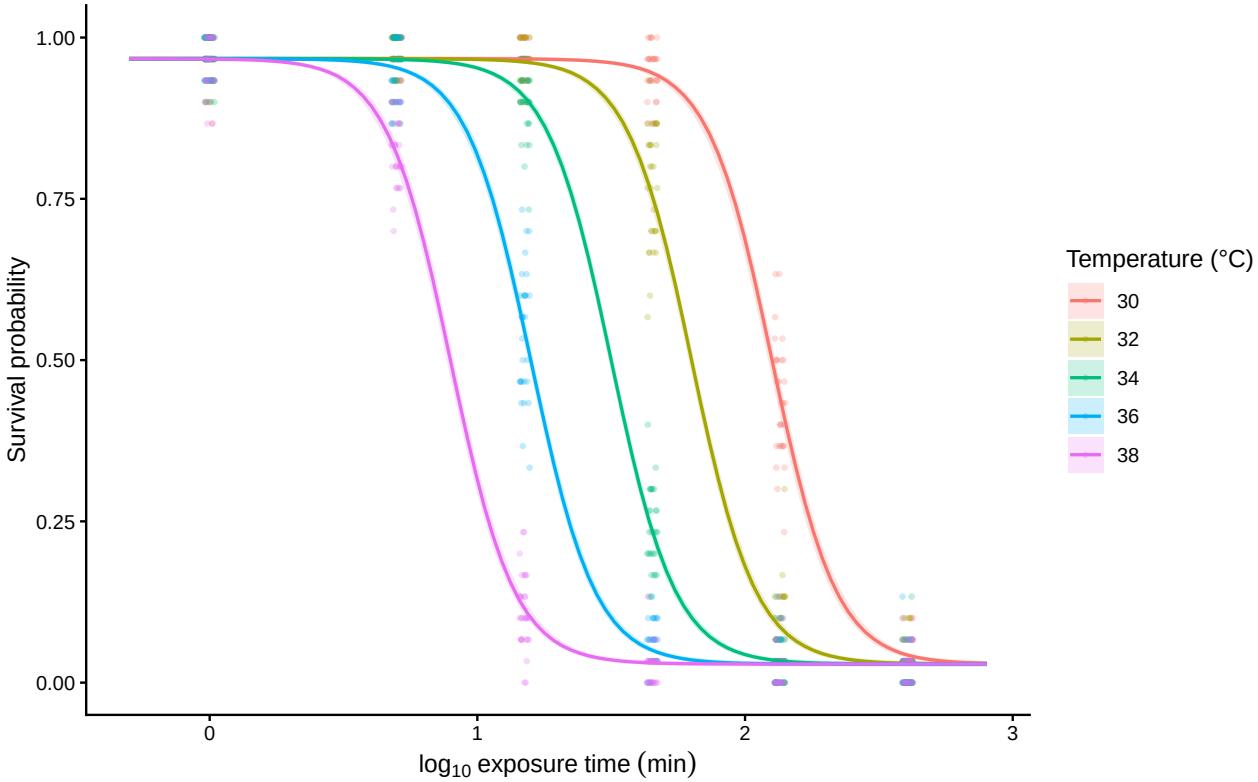

Figure S S3: Posterior fitted 4PL curves (solid lines, with shaded 95% credible bands) overlaid on the simulated binomial data (points). One panel-coloured line per assay temperature. Where the fitted curves track the simulated proportions across all five temperatures, the model has recovered the underlying dose-response surface.

1.0.4 *Deriving z and $CT_{max_{1hr}}$ from the posterior*

The whole point of fitting the joint Bayesian 4PL is that the classical TLS quantities — thermal sensitivity (z) and the critical thermal limit at a reference duration ($CT_{max_{1hr}}$) — fall out as one-line transformations of the posterior we just fit. We never refit the model; we just push the posterior draws of $(\ell, u, k, \beta_0, \beta_1)$ through the same closed-form expressions used to derive the simulation truth in Step 1.

```
# Helper: push posterior draws of (ell, u, k, beta0, beta1) through the closed-
# form derivations from the main text to get z and CTmax at 1 hour.
derive_z_ctmax <- function(fit) {
  posterior::as_draws_df(fit) |>                                          ①
    dplyr::mutate(
      z         = -1 / b_mid_T_c,                                         ②
      alpha     = b_mid_Intercept +                                       ③
        (1 / b_k_Intercept) *
        log((b_u_Intercept - 0.5) / (0.5 - b_ell_Intercept)) -
        b_mid_T_c * true_params$T_bar,
      CTmax_1hr = (log10(60) - alpha) / b_mid_T_c                         ④
```

Table S S3: Posterior medians and 95% credible intervals for thermal sensitivity (z) and the critical thermal limit at a 1-hour reference duration ($CT_{max_{1hr}}$), derived directly from the joint Bayesian 4PL fit. Both quantities recover the simulation truth under both binomial and beta-binomial likelihoods.

Quantity	Truth	Bin. median	Bin. lower	Bin. upper	BB median	BB lower	BB upper
z (°C)	6.67	6.67	6.55	6.8	6.78	6.47	7.12
CTmax at 1 hr (°C)	32.15	32.15	32.08	32.2	32.21	32.06	32.37

```
    ) |>
    dplyr::select(z, CTmax_1hr)                                                    ⑤
}

# Apply to both fits to get posterior draws of (z, CTmax_1hr).
draws_bin_d <- derive_z_ctmax(fit_bin)
draws_bb_d  <- derive_z_ctmax(fit_bb)
```

① Pull the joint posterior of $(\ell, u, k, \beta_0, \beta_1)$ from the brms fit as a data frame, with one row per posterior draw.
② For each draw, compute thermal sensitivity $z = -1/\beta_1$.
③ Compute the asymmetry-corrected intercept $\alpha = \beta_0 + (1/k)\log((u-0.5)/(0.5-\ell)) - \beta_1 \bar{T}$ used in the closed-form CT_{max} expression.
④ Compute $CT_{max_{1hr}} = (\log_{10} 60 - \alpha)/\beta_1$, the temperature at which median lethal time equals 1 hour.
⑤ Keep only the two derived quantities — the posterior is now a draws-by-2 tibble.

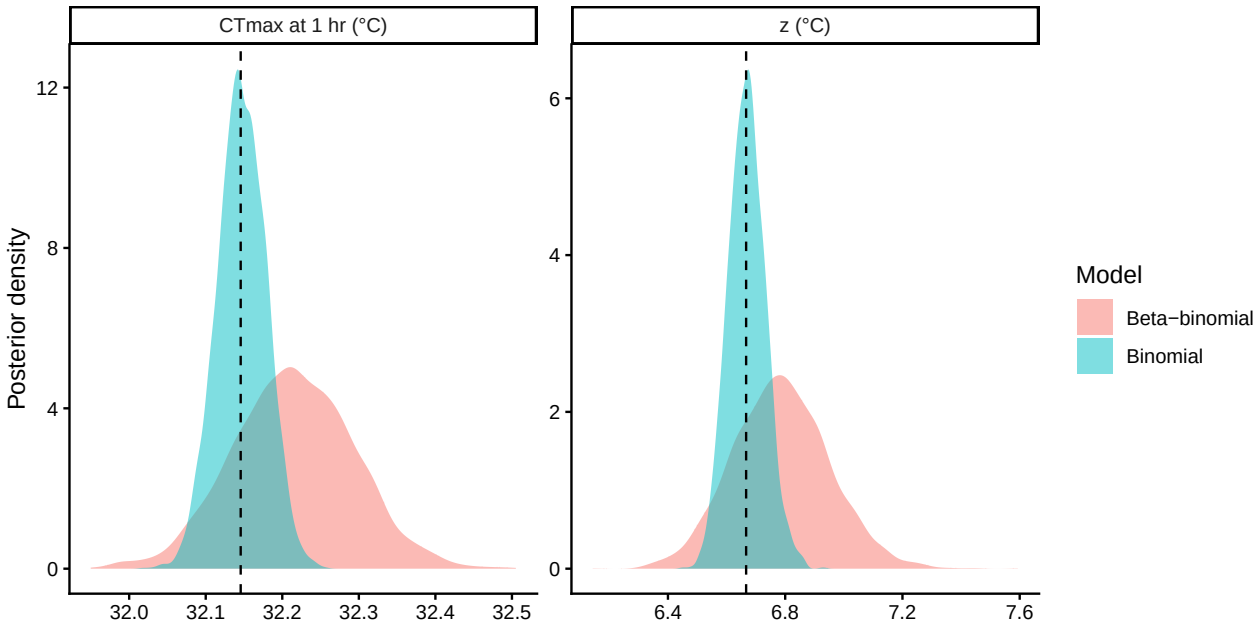

Figure S S4: Posterior distributions for thermal sensitivity (z) and $CT_{max_{1hr}}$ from the binomial fit (blue) and beta-binomial fit (orange). Vertical dashed lines mark the simulation truth. Both posteriors are tightly concentrated around the truth, illustrating that the joint Bayesian fit recovers the classical TLS quantities with calibrated uncertainty without any additional regression step.

The two derived quantities recover their simulation truth in both fits (Table S S3, Figure S S4), with credible intervals that reflect joint uncertainty in all four 4PL parameters and the temperature slope. Crucially, **we did not run a second regression** to get z and $CT_{max_{1hr}}$. They are properties of the same posterior we already have. This is the key practical advantage of the joint approach over the two-step pipeline: every classical TLS quantity is one transformation away from the fit we have already run, and every quantity inherits the same calibrated uncertainty.

Given that we have the full distribution of z and $CT_{max_{1hr}}$, we can now report on uncertainty in their values *very* easily. All we need to do is take the standard deviation of this distribution for the standard error or we can even summaise with quantiles as follows:

```
summarise_draws <- function(x) {
  q <- quantile(x, c(0.025, 0.5, 0.975), names = FALSE)
  tibble::tibble(
    mean   = mean(x),
    sd     = sd(x),
    median = q[2],
    lower  = q[1],
    upper  = q[3]
  )
}

uncert_summary <- dplyr::bind_rows(
  summarise_draws(draws_bin_d$z)          |>
```

Table S S4: Posterior summaries of z and $CT_{max_{1hr}}$ from the joint Bayesian 4PL: posterior mean, standard deviation, median, and 95% credible interval (2.5% and 97.5% quantiles), reported separately for the binomial and beta-binomial fits.

Quantity	Likelihood	Mean	SD	Median	Lower	Upper
z (°C)	Binomial	6.67	0.0629	6.67	6.55	6.8
CTmax at 1 hr (°C)	Binomial	32.15	0.032	32.15	32.08	32.21
z (°C)	Beta-binomial	6.79	0.1643	6.78	6.47	7.12
CTmax at 1 hr (°C)	Beta-binomial	32.21	0.0793	32.21	32.06	32.37

```
    dplyr::mutate(quantity = "z (°C)",             likelihood = "Binomial"),
  summarise_draws(draws_bin_d$CTmax_1hr) |>
    dplyr::mutate(quantity = "CTmax at 1 hr (°C)", likelihood = "Binomial"),
  summarise_draws(draws_bb_d$z)          |>
    dplyr::mutate(quantity = "z (°C)",             likelihood = "Beta-binomial"),
  summarise_draws(draws_bb_d$CTmax_1hr)  |>
    dplyr::mutate(quantity = "CTmax at 1 hr (°C)", likelihood = "Beta-binomial")
) |>
  dplyr::select(quantity, likelihood, mean, sd, median, lower, upper)

# Scalars for inline reporting in the next paragraph.
sd_z_bin  <- round(sd(draws_bin_d$z), 2)
sd_z_bb   <- round(sd(draws_bb_d$z), 2)
sd_ct_bin <- round(sd(draws_bin_d$CTmax_1hr), 2)
sd_ct_bb  <- round(sd(draws_bb_d$CTmax_1hr), 2)
ci_z_bin  <- round(diff(quantile(draws_bin_d$z, c(0.025, 0.975), names = FALSE)), 2)
ci_z_bb   <- round(diff(quantile(draws_bb_d$z, c(0.025, 0.975), names = FALSE)), 2)
ci_ct_bin <- round(diff(quantile(draws_bin_d$CTmax_1hr, c(0.025, 0.975), names = FALSE)), 2)
ci_ct_bb  <- round(diff(quantile(draws_bb_d$CTmax_1hr, c(0.025, 0.975), names = FALSE)), 2)

uncert_summary |>
  dplyr::rename(
    Quantity   = quantity,
    Likelihood = likelihood,
    Mean       = mean,
    SD         = sd,
    Median     = median,
    Lower      = lower,
    Upper      = upper
  ) |>
  tinytable::tt(digits = 3)
```

In short, both z and $CT_{max_{1hr}}$ are estimated with high precision and their credible intervals come straight from the joint posterior. If we fit this model using frequentist approaches we could do the same and use the delta-method, or bootstrapping to get uncertainty.

Table S S5: Point estimates (two-stage) and posterior medians (joint Bayesian) for thermal sensitivity (z) and $CT_{max_{1hr}}$, recovered by each of four estimator–dataset combinations from the same simulated datasets. Truth values are the simulation parameters.

Quantity	Truth	Two-stage (binomial)	Joint Bayesian (binomial)	Two-stage (beta-binomial)
z (°C)	6.67	6.9	6.67	6.99
CTmax at 1 hr (°C)	32.15	32	32.15	32.14

Under the binomial likelihood, the standard error (or SD of posterior) for z is 0.06 °C with a 95% credible interval 0.25 °C wide; $CT_{max_{1hr}}$ has SD 0.03 °C and a 95% credible interval width of 0.12 °C.

The beta-binomial fit is slightly wider on both quantities (SD on z: 0.16 °C; on $CT_{max_{1hr}}$: 0.08 °C) because ϕ has absorbed the extra-binomial variance and that uncertainty propagates honestly into the derived quantities — none of which falls out of the two-stage pipeline without an additional bootstrap or delta-method step.

> **!** Important
>
> It is always important to evaluate whether a binomial and a beta-binomial model better fits real data. The extra dispersion, typical of real biological data, should be estimated when it's present as it has important consequences for inferences.

1.0.5 *Comparing the two pipelines*

1.0.5.1 Point-estimate agreement on the same data

Table S S5 puts the two-stage point estimates next to the joint Bayesian posterior medians for both simulated datasets.

All four estimators land on essentially the same point estimates for z and $CT_{max_{1hr}}$, though we notice the two stage is biased upwards slightly, and all sit close to the simulation truth. The joint Bayesian 4PL gives the same answer the classical pipeline gives — it is *not* in disagreement with the field's existing point estimates, it reproduces them.

1.0.6 *Extending the model: temperature-varying shape parameters*

The closed-form derivations of z and $CT_{max_{1hr}}$ assumed that ℓ, u and k are scalars — they do not depend on T, so the asymmetry correction $\frac{1}{k}\log\frac{u-0.5}{0.5-\ell}$ is a single number rather than a function of temperature. This is convenient but not required. If ℓ, u and k also vary with temperature, then we need to predict how each of these parameters changes with temperature and apply a temperature-varying correction factor. Linear effects of temperature on the shape parameters of the dose-response curve are mechanistically interpretable and biologically plausible. Three concrete examples come to mind:

- `u ~ T_c` lets the upper asymptote (baseline survival at very short exposures) decline with rising assay temperature. This may be quite a sensible assumption if brief exposures at high T cause some immediate mortality.
- `ell ~ T_c` lets the residual surviving fraction at very long exposures depend on T. While this seems less plausible, if there is individual variation (which seems likely) some fraction may still survive, partiuclarly at less extreme temperatures, which could shift the lower baseline across each temperature treatment.
- `k ~ T_c` lets the dose-response steepness depend on T. Biologically this is the most interesting case: protein denaturation kinetics often look more threshold-like (sharper k) near a protein's melting temperature, so the slope at extreme assay temperatures may be steeper than at moderate ones and there is some suggestions that this could be true (Jørgensen *et al.* 2021; Kingsolver & Umbanhowar 2018).

When any shape parameter depends on T the asymmetry correction itself becomes a function of T:

$$\log_{10}\mathrm{LT50}(T) \;=\; \beta_0 + \beta_1\,(T-\bar{T}) \;+\; \tfrac{1}{k(T)}\log\tfrac{u(T)-0.5}{0.5-\ell(T)}. \tag{S3}$$

The midpoint piece $\beta_0+\beta_1(T-\bar{T})$ is still a straight line in T, but the asymmetry-correction piece now varies with T through $k(T), u(T), \ell(T)$. Their sum is therefore non-linear in T. Two consequences follow, and they affect z and $CT_{max_{1hr}}$ differently: z becomes a *function* of temperature (no longer a single slope but a local value at each T). In contrast, $CT_{max_{1hr}}$ remains a *single* number but its value just shifts because the underlying curve is now bent rather than straight.

- **z becomes a function of temperature.** Since z is defined by the local rate of change of $\log_{10}\mathrm{LT50}$ with T, $z(T) = -1\Big/\frac{d\log_{10}\mathrm{LT50}(T)}{dT}$. In other words, the local slope of $\log_{10}\mathrm{LT50}$ at any given T varies across the assay range because the shape of the dose-response curve changes with T. This curvature leads to the infinitesimal rates (local slopes) changing which affects the overall sensitivity, z. In these cases, it's better to estimate the local slope first. An overall z can then be obtained by averaging the local slopes if a single summary is needed (pooling the posteriors).
- **CT_{max} has no closed form.** We now need to solve $\log_{10}\mathrm{LT50}(T) = \log_{10} t_{\mathrm{ref}}$ for T per posterior draw — recovered numerically by predicting $\log_{10}\mathrm{LT50}(T)$ from the model on a fine T grid and inverting with `approx()`. The output is the same kind of posterior we already have, just produced numerically rather than algebraically.

1.0.6.1 A numerical recipe for z and CT_{max}

The simplest way to recover z and $CT_{max_{1hr}}$ from a fitted model — whether shape parameters are constant or vary with T — is to **predict survival probabilities from the model** on a fine (T, t) grid and interpolate. We can use `tidybayes::add_epred_draws()` for the prediction and base R's `approx()` for the interpolation. The same code runs whether the fit has constant shape parameters or shape parameters that vary with T.

```
# Temperatures at which we'll evaluate z. When shape parameters depend on T,
# z is itself a function of T, so we compute it at each of the original assay
# temperatures and report the full set rather than a single number.
```

```r
T_eval <- temps
half_step <- 0.5  # half-width (°C) of the central finite difference window

# Build a (T, t) grid covering the assay window. The columns log10_t, T_c, and
# n must match the variables the fitted model expects on its input side - they
# are exactly the same as the columns of `design` used to fit the model.
pred_grid <- tidyr::expand_grid(
  T = seq(min(temps) - 1, max(temps) + 1, by = 0.25),
  t = 10 ^ seq(log10(0.5), log10(800), length.out = 200)
) |>
  dplyr::mutate(
    log10_t = log10(t),
    T_c     = T - true_params$T_bar,
    n       = 1
  )

# Now, from the fitted model, draw posterior expected survival probabilities
# at every point in the (T, t) grid. `add_epred_draws()` returns a long tibble
# with one row per (grid point × posterior draw); the column .epred is the
# predicted survival probability under that draw. ndraws keeps it manageable.
epred <- pred_grid |>
  tidybayes::add_epred_draws(fit_bin, ndraws = 500)

# For each (draw, T) we find the duration t at which predicted survival
# crosses 0.5 - that's log10(LT50)(T) for that draw. Linear interpolation
# through the grid via approx() avoids any need for root-finding.
lt50_T <- epred |>
  dplyr::group_by(.draw, T) |>
  dplyr::summarise(
    log10_lt50 = approx(.epred, log10_t, xout = 0.5)$y,
    .groups    = "drop"
  )

# Per posterior draw, compute z at each evaluation temperature in T_eval via
# a central finite difference on the lt50_T grid. For each T_at we approx()
# log10(LT50) at T_at ± half_step and divide the difference by the full step
# (2 * half_step, in °C) to get the slope; z = -1 / slope.
z_per_draw <- purrr::map_dfr(T_eval, function(T_at) {
  lt50_T |>
    dplyr::group_by(.draw) |>
    dplyr::summarise(
      T_at = T_at,
      z    = -1 / ((approx(T, log10_lt50, xout = T_at + half_step)$y -
                    approx(T, log10_lt50, xout = T_at - half_step)$y) /
                   (2 * half_step)),
      .groups = "drop"
    )
```

```
})

# Per posterior draw, CTmax_1hr is the temperature at which log10(LT50) =
# log10(60). It is a single number per draw (does not depend on T_eval).
ctmax_per_draw <- lt50_T |>
  dplyr::group_by(.draw) |>
  dplyr::summarise(
    CTmax_1hr = approx(log10_lt50, T, xout = log10(60))$y,
    .groups   = "drop"
  )

# Posterior summary of z at each evaluation temperature: median and 95% CrI.
z_per_draw |>
  dplyr::group_by(T_at) |>
  dplyr::summarise(
    z_med = median(z,         na.rm = TRUE),
    z_lo  = quantile(z,         0.025, na.rm = TRUE),
    z_hi  = quantile(z,         0.975, na.rm = TRUE),
    .groups = "drop"
  )
```

```
# A tibble: 5 x 4
   T_at z_med  z_lo  z_hi
  <dbl> <dbl> <dbl> <dbl>
1    30  6.67  6.56  6.79
2    32  6.67  6.56  6.79
3    34  6.67  6.56  6.79
4    36  6.67  6.56  6.79
5    38  6.67  6.56  6.79
```

```
# Posterior summary of CTmax_1hr.
ctmax_per_draw |>
  dplyr::summarise(
    CTmax_1hr_med = median(CTmax_1hr, na.rm = TRUE),
    CTmax_1hr_lo  = quantile(CTmax_1hr, 0.025, na.rm = TRUE),
    CTmax_1hr_hi  = quantile(CTmax_1hr, 0.975, na.rm = TRUE)
  )
```

```
# A tibble: 1 x 3
  CTmax_1hr_med CTmax_1hr_lo CTmax_1hr_hi
          <dbl>        <dbl>        <dbl>
1          32.1         32.1         32.2
```

The posterior medians of z at each of the five assay temperatures are essentially identical to one another, and the common value matches the closed-form posterior median of z from Section 1.0.4 — exactly as expected for a fit with linear `mid(T)` and constant shape parameters, where z does

not depend on temperature. This flatness is the sanity check; if the same recipe is applied to a fit that lets the shape parameters vary with T (e.g. `k ~ T_c`), the same output will show z varying across the assay range, as it should. $CT_{max_{1hr}}$ likewise reproduces the closed-form posterior in `draws_bin_d` from Section 1.0.4.

1.0.6.2 Demonstrating changes with temperature-varying shape parameters

To make the points above concrete, we re-simulate beta-binomial data on the same factorial design as the simple case, but now both the dose-response steepness k and the upper asymptote u vary linearly with assay temperature. We then fit the joint Bayesian 4PL with the corresponding `bf()` formula and apply the same predict-and-interpolate recipe.

```
# (Re-defined here so this chunk can be run independently of the earlier
# simulation. Same values as the original `sim-design` chunk in @sec-sim-step2.)
temps     <- c(30, 32, 34, 36, 38)             # assay temperatures (°C)
durations <- c(1, 5, 15, 45, 135, 405)         # exposure durations (min)
n_rep     <- 30
n_per_rep <- 30
phi       <- 5                                  # beta-binomial dispersion

design <- tidyr::expand_grid(
  T   = temps,
  t   = durations,
  rep = seq_len(n_rep)
) |>
  dplyr::mutate(
    log10_t = log10(t),
    T_c     = T - 34,                           # mean-centred T (T_bar = 34)
    n       = n_per_rep
  )

# Truth is defined on a constrained raw scale. This keeps ell in [0, 0.49],
# u in [0.51, 1], and k positive at every assay temperature. Those constraints
# avoid the label-switching and invalid-curve geometry that make identity-scale
# 4PL fits mix poorly when u and k vary with temperature.
true_params_ext <- list(
  ell_raw  = qlogis(0.05 / 0.49),          # ell = 0.05
  u_raw_0  = qlogis((0.92 - 0.51) / 0.49), # u = 0.92 at T_bar
  u_raw_T  = -0.20,                        # u declines as T rises
  log_k_0  = log(8),                       # k = 8 at T_bar
  log_k_T  = 0.10,                         # k rises as T rises
  m_beta0  = 1.5,
  m_beta1  = -0.15,
  T_bar    = 34
)

# Build per-cell true survival probabilities and draw beta-binomial counts.
```

```r
# (mutate keeps all columns of `design`; mid and p_true from the original
# sim are overwritten in place with the new T-varying values.)
sim_bb_ext <- design |>
  dplyr::mutate(
    ell    = 0.49 * plogis(true_params_ext$ell_raw),
    u      = 0.51 + 0.49 * plogis(true_params_ext$u_raw_0 +
                                  true_params_ext$u_raw_T * T_c),
    k      = exp(true_params_ext$log_k_0 + true_params_ext$log_k_T * T_c),
    mid    = true_params_ext$m_beta0 + true_params_ext$m_beta1 * T_c,
    p_true = ell +
             (u - ell) /
             (1 + exp(k * (log10_t - mid))),
    p_draw = rbeta(dplyr::n(), p_true * phi, (1 - p_true) * phi),
    y      = rbinom(dplyr::n(), size = n, prob = p_draw)
  )
```

Lets have a look at the data along with truth (Figure S S5):

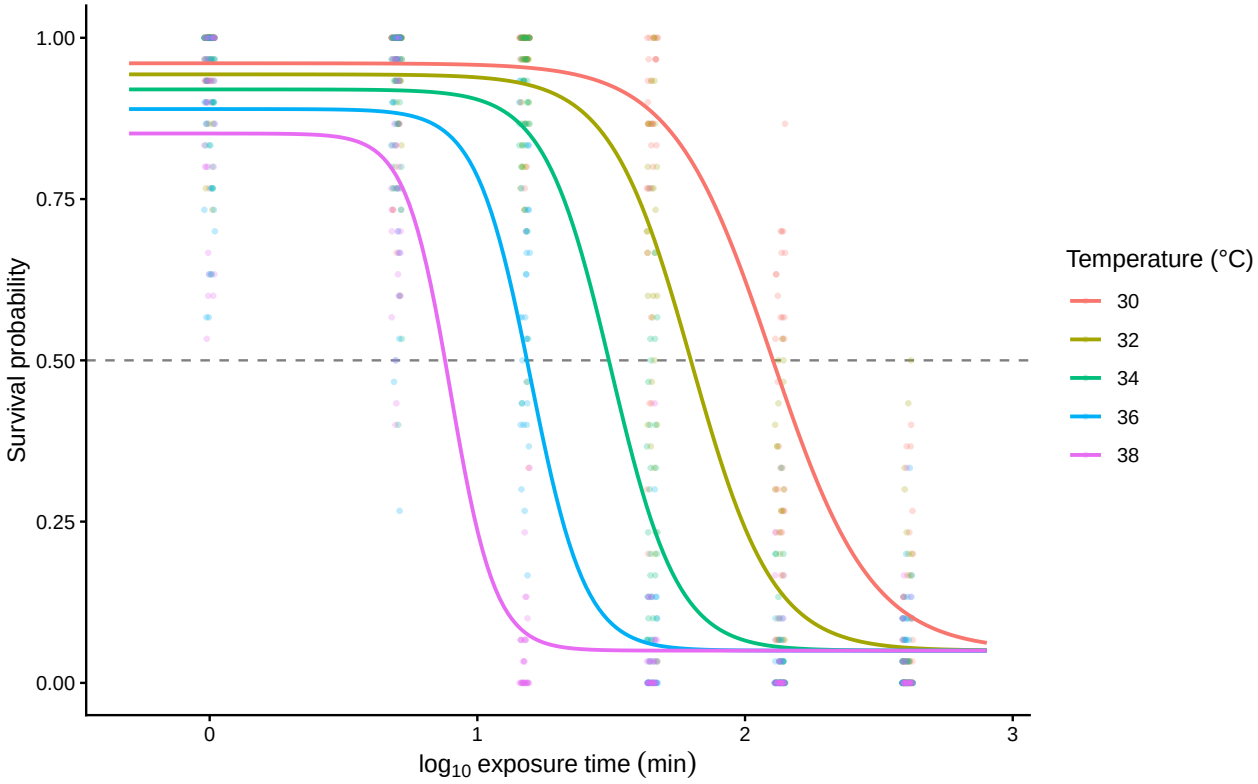

Figure S S5: True 4PL dose-response surface (solid lines) for the extended simulation, where k rises and u falls linearly with assay temperature, overlaid on the simulated beta-binomial counts (points). One coloured curve and a corresponding cloud of points per assay temperature; the horizontal dashed line marks 50% survival.

Now that we have the simulated data, let's fit the model. The important change from the simple-model fit is that the four 4PL parameters are no longer fit directly on the identity scale — each

one needs a different transformation because each has a different constraint:

Parameter	Natural-scale constraint	Transform used
`ell` (lower asymptote)	$[0, 0.49]$	`0.49 * inv_logit(ellraw)` — squashes raw real to $(0, 0.49)$
`u` (upper asymptote)	$[0.51, 1]$	`0.51 + 0.49 * inv_logit(uraw)` — squashes raw real to $(0.51, 1)$
`k` (slope)	> 0	`exp(logk)` — squashes raw real to $(0, \infty)$
`mid` (midpoint)	unconstrained	identity (no transform)

The asymptotes use the **disjoint** intervals $[0, 0.49]$ and $[0.51, 1]$ (rather than each covering all of $[0, 1]$) to prevent MCMC sampling issues where upper and lower asymptotes flip during MCMC sampling which can cause convergence issues — to help here we constrain them and estimate on the logit scale. The 0.02 gap at $p = 0.5$ guarantees $u > \ell$ always, so the chains can't oscillate — which matters here because u and ℓ can vary across temperatures. The cost is hard-bounding the asymptotes a few percentage points away from 0.5 — fine for survival data where genuine LT_{50} assays don't sit at the boundary anyway. We also constrain `k` to be positive, but on a log scale via `exp(logk)`. Because u, ℓ and `k` have different transformations and meanings, we apply each transformation per parameter inside the formula before they are combined — as opposed to a single link function on the linear predictor as is typical in GLMMs.

We just need to remember in our fits that these values are transformed so we need to back-transform to make sense of the coefficients.

```
# Disable Quarto chunk caching here so brms's own `file = ...` /
# `file_refit = "on_change"` mechanism is the sole source of truth for whether
# to refit. Without this, the project default `cache: true` can hold an old
# fit object even after the formula or priors change.

# Same 4PL likelihood as before, but with constrained transformations:
#   ell = 0.49 * inv_logit(ellraw)
#   u   = 0.51 + 0.49 * inv_logit(uraw)
#   k   = exp(logk)
# This guarantees ell < 0.5, u > 0.5, u > ell, and k > 0.
bform_ext <- bf(
  y | trials(n) ~
    0.49 * inv_logit(ellraw) +
      ((0.51 + 0.49 * inv_logit(uraw)) -
         0.49 * inv_logit(ellraw)) /
      (1 + exp(exp(logk) * (log10_t - mid))),
  ellraw ~ 1,
  uraw   ~ T_c,
  logk   ~ T_c,
  mid ~ T_c,
  nl = TRUE,
```

```
  family = beta_binomial(link = "identity")
)

priors_ext <- c(
  brms::prior_string(
    sprintf("normal(%f, 0.50)", true_params_ext$ell_raw),
    nlpar = "ellraw", coef = "Intercept"
  ),
  brms::prior_string(
    sprintf("normal(%f, 0.50)", true_params_ext$u_raw_0),
    nlpar = "uraw", coef = "Intercept"
  ),
  brms::prior_string(
    sprintf("normal(%f, 0.10)", true_params_ext$u_raw_T),
    nlpar = "uraw", coef = "T_c"
  ),
  brms::prior_string(
    sprintf("normal(%f, 0.35)", true_params_ext$log_k_0),
    nlpar = "logk", coef = "Intercept"
  ),
  brms::prior_string(
    sprintf("normal(%f, 0.05)", true_params_ext$log_k_T),
    nlpar = "logk", coef = "T_c"
  ),
  prior(normal(1.5, 0.40), nlpar = "mid", coef = "Intercept"),
  prior(normal(-0.15, 0.04), nlpar = "mid", coef = "T_c")
)

fit_bb_ext <- brm(
  bform_ext, data = sim_bb_ext, prior = priors_ext,
  chains = 4, iter = 2000, cores = 4, seed = 123,
  control = list(adapt_delta = 0.95, max_treedepth = 12),
  file = file.path(models_dir, "sim_4pl_ext_betabinomial_reparam"),
  file_refit = "on_change"
)
```

The printed model summary is now mainly a convergence check because the coefficients live on raw transformed scales. We still want Rhat near 1 and reasonable effective sample sizes, but recovery is easier to inspect after transforming the posterior draws back to natural 4PL parameters.

```
# Force this chunk to re-run on every render so the printed summary always
# reflects the current `fit_bb_ext` object (the project default `cache: true`
# would otherwise hold onto a stale snapshot when the model is refit upstream).
fit_bb_ext
```

```
 Family: beta_binomial
```

```
  Links: mu = identity
Formula: y | trials(n) ~ 0.49 * inv_logit(ellraw) + ((0.51 + 0.49 * inv_logit(uraw)) - 0.49 * i
         ellraw ~ 1
         uraw ~ T_c
         logk ~ T_c
         mid ~ T_c
   Data: sim_bb_ext (Number of observations: 900)
  Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
         total post-warmup draws = 4000

Regression Coefficients:
                 Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
ellraw_Intercept    -2.31      0.11    -2.54    -2.09 1.00     3056     3207
uraw_Intercept       1.56      0.10     1.37     1.75 1.00     2753     2571
uraw_T_c            -0.22      0.03    -0.28    -0.15 1.00     3307     2825
logk_Intercept       2.06      0.07     1.93     2.21 1.00     2700     2474
logk_T_c             0.15      0.02     0.11     0.19 1.00     3038     2790
mid_Intercept        1.50      0.01     1.48     1.53 1.00     3973     3031
mid_T_c             -0.14      0.01    -0.15    -0.13 1.00     4236     3099

Further Distributional Parameters:
    Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
phi     4.75      0.36     4.08     5.49 1.00     3381     2977

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

Now we can check the changes in z and $CT_{max_{1hr}}$ from having temperature impact on multiple parameters of the 4PL using our workflow in the previous section.

```
# Predict-and-interpolate recipe applied to the extended fit. The code is
# identical to @sec-joint-extend-recipe - only the fit changes.
pred_grid_ext <- tidyr::expand_grid(
  T = seq(min(temps) - 1, max(temps) + 1, by = 0.25),
  t = 10 ^ seq(log10(0.5), log10(800), length.out = 200)
) |>
  dplyr::mutate(
    log10_t = log10(t),
    T_c     = T - true_params_ext$T_bar,
    n       = 1
  )

epred_ext <- pred_grid_ext |>
  tidybayes::add_epred_draws(fit_bb_ext, ndraws = 500)

lt50_T_ext <- epred_ext |>
```

Table S S7: Natural-scale parameter recovery from the constrained temperature-varying 4PL. The posterior is transformed from the raw `ellraw`, `uraw`, and `logk` coefficients back to the biologically interpretable asymptotes, steepness, and midpoint at each assay temperature; ϕ is the beta-binomial precision parameter used to generate the overdispersed counts.

Parameter	Temperature (°C)	Truth	Posterior median	Lower	Upper
ell	30	0.05	0.0443	0.0359	0.0539
k	30	5.363	4.3952	3.7429	5.1646
mid	30	2.1	2.0751	2.0288	2.1233
u	30	0.961	0.9603	0.9466	0.9716
ell	32	0.05	0.0443	0.0359	0.0539
k	32	6.55	5.8805	5.2081	6.673
mid	32	1.8	1.7893	1.7585	1.8221
u	32	0.943	0.9412	0.9282	0.953
ell	34	0.05	0.0443	0.0359	0.0539
k	34	8	7.8697	6.8926	9.1089
mid	34	1.5	1.5043	1.4804	1.5293
u	34	0.92	0.9148	0.901	0.9272
ell	36	0.05	0.0443	0.0359	0.0539
k	36	9.771	10.5314	8.8045	12.9561
mid	36	1.2	1.2187	1.1896	1.2488
u	36	0.89	0.8793	0.859	0.898
ell	38	0.05	0.0443	0.0359	0.0539
k	38	11.935	14.0821	10.9943	18.8182
mid	38	0.9	0.9331	0.8903	0.978
u	38	0.852	0.8355	0.8005	0.8673
phi	all	5	4.7371	4.0814	5.4931

```
  dplyr::group_by(.draw, T) |>
  dplyr::summarise(
    log10_lt50 = approx(.epred, log10_t, xout = 0.5)$y,
    .groups    = "drop"
  )

z_per_draw_ext <- purrr::map_dfr(T_eval, function(T_at) {
  lt50_T_ext |>
    dplyr::group_by(.draw) |>
    dplyr::summarise(
      T_at = T_at,
      z    = -1 / ((approx(T, log10_lt50, xout = T_at + half_step)$y -
                    approx(T, log10_lt50, xout = T_at - half_step)$y) /
                   (2 * half_step)),
      .groups = "drop"
    )
})

ctmax_per_draw_ext <- lt50_T_ext |>
  dplyr::group_by(.draw) |>
  dplyr::summarise(
    CTmax_1hr = approx(log10_lt50, T, xout = log10(60))$y,
    .groups   = "drop"
  )

# Posterior summaries: z at each assay temperature, and CTmax_1hr.
z_summary_ext <- z_per_draw_ext |>
  dplyr::group_by(T_at) |>
  dplyr::summarise(
    z_med = median(z, na.rm = TRUE),
    z_lo  = quantile(z, 0.025, na.rm = TRUE),
    z_hi  = quantile(z, 0.975, na.rm = TRUE),
    .groups = "drop"
  )

ctmax_summary_ext <- ctmax_per_draw_ext |>
  dplyr::summarise(
    CTmax_1hr_med = median(CTmax_1hr, na.rm = TRUE),
    CTmax_1hr_lo  = quantile(CTmax_1hr, 0.025, na.rm = TRUE),
    CTmax_1hr_hi  = quantile(CTmax_1hr, 0.975, na.rm = TRUE)
  )

z_summary_ext
```

```
# A tibble: 5 x 4
   T_at z_med  z_lo  z_hi
  <dbl> <dbl> <dbl> <dbl>
```

```
1    30  6.79  6.41  7.31
2    32  6.82  6.44  7.36
3    34  6.85  6.47  7.41
4    36  6.88  6.48  7.44
5    38  6.90  6.49  7.47
```

```
ctmax_summary_ext
```

```
# A tibble: 1 x 3
  CTmax_1hr_med CTmax_1hr_lo CTmax_1hr_hi
          <dbl>        <dbl>        <dbl>
1          32.0         31.8         32.2
```

In the simple-model fit (Section 1.0.4), z was essentially the same value at every assay temperature. In this extended fit, the posterior medians of z vary across temperature (albeit slightly): a direct consequence of the temperature-dependent k and u, which bend the $\log_{10}\mathrm{LT50}(T)$ curve.

In contrast, $CT_{max_{1hr}}$ remains a single number per posterior draw, and in this simulation it actually lands very close to the simple-model value (~32 °C): the asymmetry correction is near zero at the temperature where the LT50 curve crosses 1 hour, so $CT_{max_{1hr}}$ is locally almost insensitive to the T-varying shape parameters. With more pronounced T-dependence in k and u, or with shape parameters further from the symmetric ($\ell + u \approx 1$) regime, $CT_{max_{1hr}}$ would shift more visibly.

If we now want to get an adjusted z that averages across these temperatures we can do that as follows:

```
# For each posterior draw, average the five local z(T_eval) values into
# a single adjusted z; then summarise across draws.
adjusted_z_ext <- z_per_draw_ext |>
  dplyr::group_by(.draw) |>
  dplyr::summarise(z_mean = mean(z, na.rm = TRUE), .groups = "drop")

adjusted_z_ext |>
  dplyr::summarise(
    z_med = median(z_mean,   na.rm = TRUE),
    z_lo  = quantile(z_mean, 0.025, na.rm = TRUE),
    z_hi  = quantile(z_mean, 0.975, na.rm = TRUE)
  )
```

```
# A tibble: 1 x 3
  z_med  z_lo  z_hi
  <dbl> <dbl> <dbl>
1  6.85  6.46  7.40
```

Compared to the simple-model $z = 6.67$ °C (where shape parameters did not vary with temperature), this adjusted z is 6.85 °C with 95% credible interval [6.46, 7.4] °C — slightly lower in point estimate, with the simple-model value sitting near the upper end of the credible interval. With stronger T-dependence in k and u, the bent $\log_{10}\mathrm{LT50}(T)$ curve would have more variation across temperatures and the two summaries would diverge more.

1.0.7 Heat injury accumulation and survival under a realistic field trace

A particularly powerful aspect of TLS models is their ability to translate dynamic temperature time-series from nature and predict how such temperatures lead to heat injury (HI) accumulation towards a threshold (e.g., LT_{50}) (Arnold *et al.* 2025; Jørgensen *et al.* 2021; Noble *et al.* 2026; Ørsted *et al.* 2022; Rezende *et al.* 2020). The joint Bayesian 4PL fit gives us not just point estimates of z and $CT_{max_{1hr}}$ but their full joint posterior, which we can push through the HI integral to obtain a posterior distribution over the predicted HI trajectory. We can also go a step further and use the *full* 4PL surface to predict the cumulative survival fraction $S_{\text{pred}}(t)$ across the trajectory — the second-step prediction discussed in the manuscript (Equation 9 of the manuscript).

Below we apply both calculations to a realistic 5-day field temperature trace using the posterior of `fit_bb` derived in Section 1.0.4.

1.0.7.1 A simulated temperature profile across 5 days

To mimic conditions a moderately heat-tolerant ectotherm might experience over a temperate-zone late-spring period, we generate 5 days of hourly temperatures with a sinusoidal diurnal cycle, ranging from 12°C overnight to a midday peak of 24°C. The critical temperature T_c — below which damage is repaired as fast as it accrues and so there is no net damage increase (Jørgensen *et al.* 2021; Ørsted *et al.* 2022) — is set to 20°C. Injury accumulates only during the warmer part of each day. The simulated organism has $CT_{max_{1hr}} \approx 32.1$°C and $z \approx 6.7$°C (from `true_params` in Section 1.0.1.1), so the daily peak at 24°C remains 8°C below the 1-hour critical limit — moderate, sub-lethal heat stress that builds cumulatively across days.

```
# Hourly temperature trace: 5 days, sinusoidal diurnal cycle
hi_hours  <- 0:(24 * 5 - 1)
hi_T_traj <- 18 + 6 * sin(2 * pi * (hi_hours - 6) / 24)   # range 12-24 °C
hi_T_c    <- 20                                            # damage threshold (°C)
```

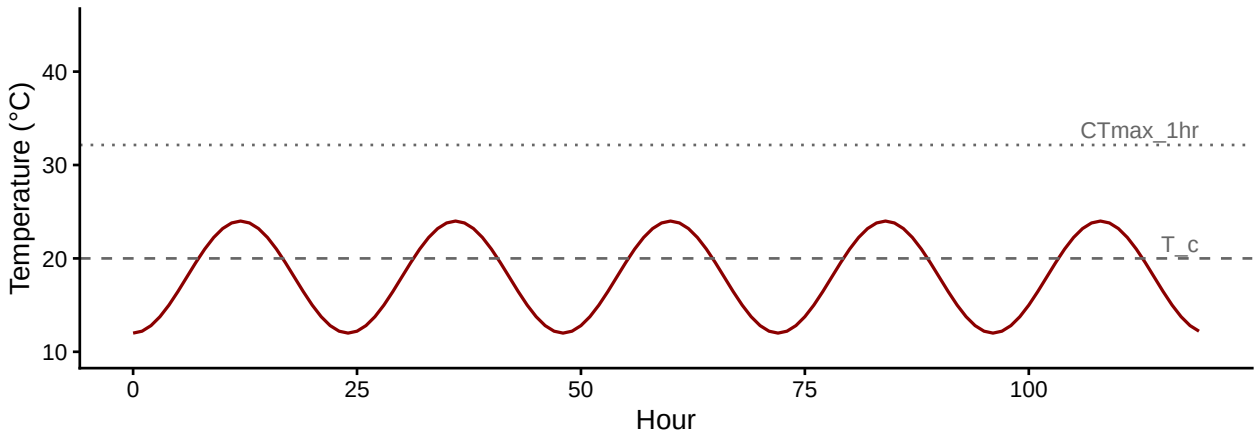

Figure S S6: 5-day diurnal temperature trace used for the heat-injury and survival predictions. The dashed line is the damage-accumulation threshold $T_c = 20$ °C; the dotted line is the simulated organism's $CT_{max_{1hr}} \approx 32.1$°C — well above the daily peaks, so accumulation is sub-lethal and gradual.

1.0.7.2 HI accumulation with full posterior uncertainty

For each posterior sample of $(z, CT_{max_{1hr}})$, we evaluate the HI integral (Equation 7 of the manuscript) across the temperature trace. Each sample produces one possible HI(t) trajectory. The posterior median and 95% credible interval at every hour will then summarise the prediction with full parameter uncertainty.

```
# Subsample 500 posterior draws from the beta-binomial fit for speed.
set.seed(20260507)
hi_draws <- posterior::as_draws_df(fit_bb) |>
  dplyr::mutate(
    z         = -1 / b_mid_T_c,
    alpha     = b_mid_Intercept +
                  (1 / b_k_Intercept) *
                  log((b_u_Intercept - 0.5) / (0.5 - b_ell_Intercept)) -
                  b_mid_T_c * true_params$T_bar,
    CTmax_1hr = (log10(60) - alpha) / b_mid_T_c
  ) |>
  dplyr::slice_sample(n = 500)

# Build the (n_hours × n_draws) rate matrix and cumsum down each column.
n_h <- length(hi_T_traj)
n_d <- nrow(hi_draws)

T_mat  <- matrix(hi_T_traj, n_h, n_d)
CT_mat <- matrix(hi_draws$CTmax_1hr, n_h, n_d, byrow = TRUE)
z_mat  <- matrix(hi_draws$z, n_h, n_d, byrow = TRUE)

# Calculate the rates of HI accumulation from our posterior samples of z and CTmax for the give
rates              <- 100 * 10 ^ ((T_mat - CT_mat) / z_mat)
rates[T_mat <= hi_T_c] <- 0
HI_traj            <- apply(rates, 2, cumsum)
HI_traj[HI_traj > 200] <- 200 # Cap at 200% because survival can't go beyond that.

# Create a summary table that takes the median and 95% CI across the posterior samples for eac
hi_summary <- tibble::tibble(
  hour   = hi_hours,
  median = apply(HI_traj, 1, median),
  lower  = apply(HI_traj, 1, quantile, probs = 0.025),
  upper  = apply(HI_traj, 1, quantile, probs = 0.975)
)
```

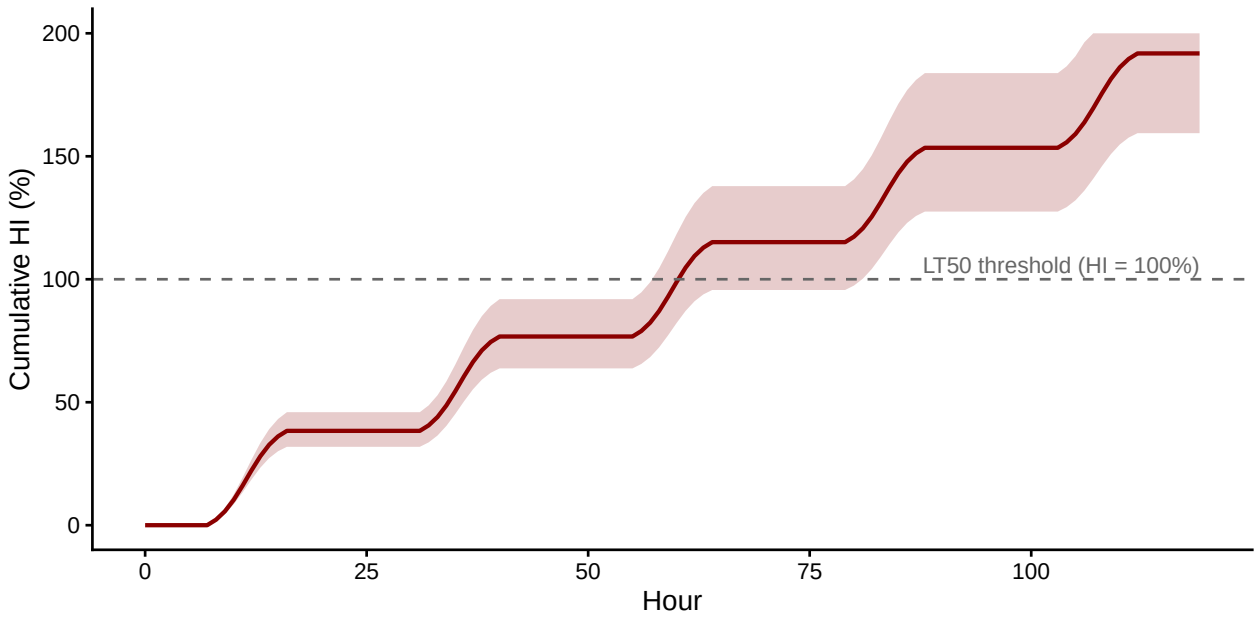

Figure S S7: Posterior cumulative heat-injury trajectory under the 5-day field trace. Solid line is the posterior median across 500 draws of $(z, CT_{max_{1hr}})$ from `fit_bb`; shaded band is the 95% credible interval. The dashed line marks the LT_{50} threshold ($HI = 100\%$), which the population is predicted to cross by day 3.

By the end of day 5 the posterior median HI is 192% (95% CI: 159% to 200%) — i.e., the population has accumulated roughly 1.9 × the lethal dose under this exposure.

1.0.7.3 Predicting survival from the full 4PL surface

The HI integral compresses the trajectory into a single threshold-crossing summary. The joint Bayesian 4PL fit also lets us predict the *full* survival fraction $S_{\text{pred}}(t)$ at every time point along the trace, by accumulating effective exposure time at a reference temperature and reading survival off the dose-response surface at the cumulative effective time (Equation 9 of the manuscript). We use $T_{\text{ref}} = \bar{T} = 34$ °C — the grand-mean temperature of the original assays — so the surface is evaluated at the most data-rich part of the dose-response.

```
T_ref <- true_params$T_bar

# For each draw, accumulate effective minutes at T_ref, then evaluate the 4PL.
S_traj <- matrix(0, n_h, n_d)
for (d in seq_len(n_d)) {
  z_d     <- hi_draws$z[d]
  ell_d   <- hi_draws$b_ell_Intercept[d]
  u_d     <- hi_draws$b_u_Intercept[d]
  k_d     <- hi_draws$b_k_Intercept[d]
  beta0_d <- hi_draws$b_mid_Intercept[d]   # midpoint at T_bar (in log10-min)

  # Effective-time increment in MINUTES per hourly bin at T_ref = T_bar.
```

```r
  inc_min     <- ifelse(hi_T_traj > hi_T_c,
                        60 * 10 ^ ((hi_T_traj - T_ref) / z_d),
                        0)
  tau_eff_min <- cumsum(inc_min)

  # 4PL surface evaluated at (T_ref, tau_eff). Note that tau_eff_min can be 0, and log10(0)is
  log10_tau   <- log10(pmax(tau_eff_min, .Machine$double.eps))
  S_traj[, d] <- ell_d + (u_d - ell_d) /
                   (1 + exp(k_d * (log10_tau - beta0_d)))
}

S_summary <- tibble::tibble(
  hour   = hi_hours,
  median = apply(S_traj, 1, median),
  lower  = apply(S_traj, 1, quantile, probs = 0.025),
  upper  = apply(S_traj, 1, quantile, probs = 0.975)
)
```

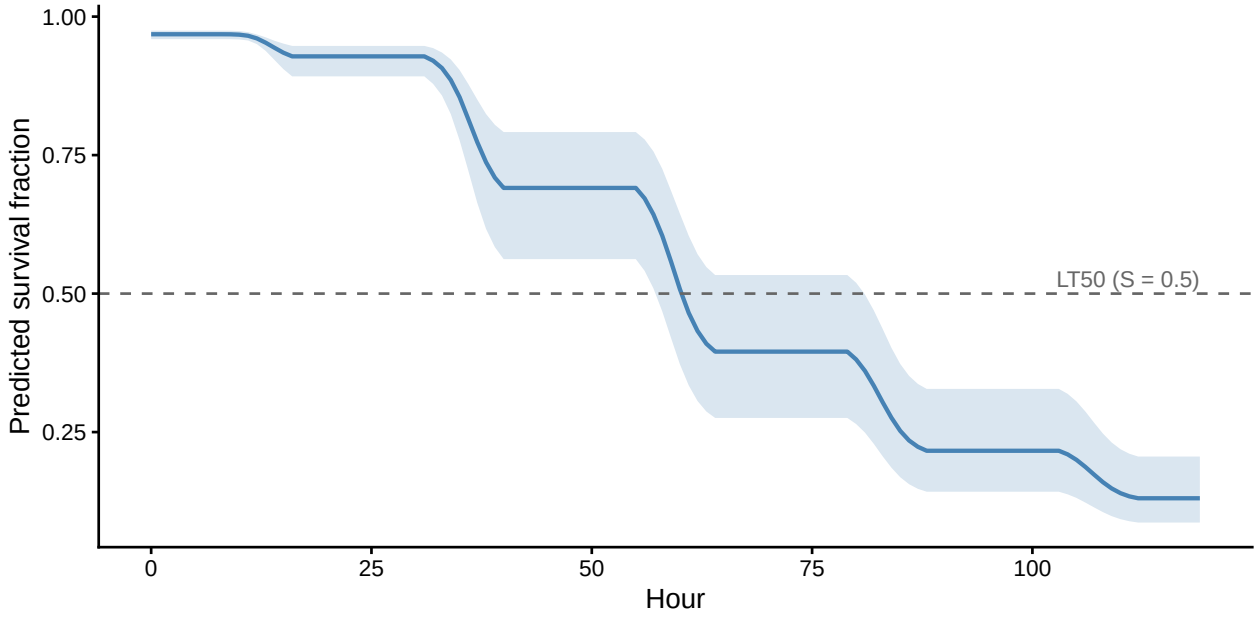

Figure S S8: Posterior predicted population-level survival fraction under the 5-day field trace, computed from the model (Equation 9 of the manuscript). Solid line is the posterior median across 500 draws of $(\ell, u, k, \beta_0, \beta_1)$ from `fit_bb`; shaded band is the 95% credible interval. The dashed line marks 50% survival, the threshold the HI integral compresses into a single crossing time.

By the end of day 5 the posterior median predicted survival is 0.13 (95% CI: 0.09 to 0.21). The two predictions tell consistent stories: $S_{\text{pred}}(t)$ falls through 0.5 at the same time that $HI(t)$ rises through 100 %, but $S_{\text{pred}}(t)$ provides the entire cumulative survival curve. Survival can then be used directly for population-dynamical models, which has been suggsted by Noble *et al.* (2026).

2 References

Arnold, P.A., Noble, D.W.A., Nicotra, A.B., Kearney, M.R., Rezende, E.L., Andrew, S.C., *et al.* (2025). A framework for modelling thermal load sensitivity across life. *Global Change Biology*, 31, e70315.

Jørgensen, L.B., Malte, H. & Overgaard, J. (2021). A unifying model to estimate thermal tolerance limits in ectotherms across static, dynamic and fluctuating exposures to thermal stress. *Scientific Reports*, 11, 12840.

Kingsolver, J.G. & Umbanhowar, J. (2018). The analysis and interpretation of critical temperatures. *Journal of Experimental Biology*, 221, jeb167858.

Noble, D.W.A., Mayfield, M.M., Hoffmann, A.A., Chen, Z.-H., Lade, S.J., Bai, X., *et al.* (2026). A systems modelling approach to predict biological responses to extreme heat. *Trends in Ecology & Evolution*, 41, 451–464.

Ørsted, M., Jørgensen, L.B. & Overgaard, J. (2022). Finding the right thermal limit: A framework to reconcile ecological, physiological and methodological aspects of CTmax in ectotherms. *Journal of Experimental Biology*, 225, jeb244514.

Rezende, E.L., Bozinovic, F., Szilágyi, A. & Santos, M. (2020). Predicting temperature mortality and selection in natural Drosophila populations. *Science*, 369, 1242–1245.

Rezende, E.L., Castañeda, L.E. & Santos, M. (2014). Tolerance landscapes in thermal ecology. *Functional Ecology*, 28, 799–809.

3 Session Information

R version 4.5.3 (2026-03-11)

Platform: aarch64-apple-darwin20

locale: C.UTF-8||C.UTF-8||C.UTF-8||C||C.UTF-8||C.UTF-8

attached base packages: *stats*, *graphics*, *grDevices*, *utils*, *datasets*, *methods* and *base*

other attached packages: *pander(v.0.6.6)*, *tinytable(v.0.16.0)*, *tidybayes(v.3.0.7)*, *posterior(v.1.6.1)*, *brms(v.2.23.0)*, *Rcpp(v.1.1.1-1.1)*, *ggplot2(v.4.0.3)*, *purrr(v.1.2.2)*, *tibble(v.3.3.1)*, *tidyr(v.1.3.2)*, *dplyr(v.1.2.1)* and *here(v.1.0.1)*

loaded via a namespace (and not attached): *svUnit(v.1.0.8)*, *tidyselect(v.1.2.1)*, *farver(v.2.1.2)*, *loo(v.2.9.0)*, *S7(v.0.2.2)*, *fastmap(v.1.2.0)*, *TH.data(v.1.1-3)*, *tensorA(v.0.36.2.1)*, *pacman(v.0.5.1)*, *digest(v.0.6.39)*, *estimability(v.1.5.1)*, *lifecycle(v.1.0.5)*, *StanHeaders(v.2.32.10)*, *processx(v.3.9.0)*, *survival(v.3.8-6)*, *magrittr(v.2.0.5)*, *compiler(v.4.5.3)*, *rlang(v.1.2.0)*, *tools(v.4.5.3)*, *yaml(v.2.3.12)*, *knitr(v.1.51)*, *labeling(v.0.4.3)*, *bridgesampling(v.1.2-1)*, *pkgbuild(v.1.4.8)*, *curl(v.7.1.0)*, *plyr(v.1.8.9)*, *RColorBrewer(v.1.1-3)*, *multcomp(v.1.4-28)*, *abind(v.1.4-8)*, *withr(v.3.0.2)*, *grid(v.4.5.3)*, *stats4(v.4.5.3)*, *colorspace(v.2.1-1)*, *xtable(v.1.8-8)*, *inline(v.0.3.21)*, *emmeans(v.2.0.3)*, *scales(v.1.4.0)*, *MASS(v.7.3-65)*, *tinytex(v.0.59)*, *cli(v.3.6.6)*, *mvtnorm(v.1.3-7)*, *rmarkdown(v.2.31)*, *generics(v.0.1.4)*, *otel(v.0.2.0)*, *RcppParallel(v.5.1.11-2)*, *reshape2(v.1.4.5)*, *rstan(v.2.32.7)*, *stringr(v.1.6.0)*, *splines(v.4.5.3)*, *bayesplot(v.1.15.0)*, *parallel(v.4.5.3)*, *matrixStats(v.1.5.0)*, *vctrs(v.0.7.3)*, *V8(v.6.0.4)*, *Matrix(v.1.7-4)*, *sandwich(v.3.1-1)*, *jsonlite(v.2.0.0)*, *callr(v.3.7.6)*, *litedown(v.0.7)*, *arrayhelpers(v.1.1-0)*, *ggdist(v.3.3.3)*, *glue(v.1.8.1)*, *codetools(v.0.2-20)*, *distributional(v.0.6.0)*, *stringi(v.1.8.7)*, *gtable(v.0.3.6)*, *QuickJSR(v.1.9.0)*, *pillar(v.1.11.1)*,

htmltools(v.0.5.9), *Brobdingnag(v.1.2-9)*, *R6(v.2.6.1)*, *rprojroot(v.2.1.1)*, *evaluate(v.1.0.5)*, *lattice(v.0.22-9)*, *backports(v.1.5.1)*, *rstantools(v.2.6.0)*, *coda(v.0.19-4.1)*, *gridExtra(v.2.3)*, *nlme(v.3.1-168)*, *checkmate(v.2.3.4)*, *mgcv(v.1.9-4)*, *xfun(v.0.57)*, *zoo(v.1.8-14)* and *pkgconfig(v.2.0.3)*